БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Кафедра интеллектуальных информационных технологий

ЛАБОРАТОРНАЯ РАБОТА №1

Алгоритмы построения отрезков

Дисциплина: Графический интерфейс
интеллектуальных систем

Технологии: Python 3.11, PyQt5

1. Цель работы

Разработать элементарный графический редактор, реализующий построение отрезков с помощью трёх алгоритмов: ЦДА (цифровой дифференциальный анализатор), целочисленного алгоритма Брезенхема и алгоритма Ву (со сглаживанием).

Редактор должен поддерживать пользовательский режим (рисование отрезков) и отладочный режим (пошаговая визуализация работы алгоритма на дискретной сетке).

2. Теоретические сведения

2.1. Алгоритм ЦДА

Цифровой дифференциальный анализатор (ЦДА) — простейший алгоритм растеризации отрезка, основанный на решении дифференциального уравнения прямой линии. Метод использует вещественную арифметику.

Принцип работы:

1) Вычисляется длина проекции отрезка: Length = max(|x2-x1|, |y2-y1|).
2) Определяются приращения: dx = (x2-x1)/Length, dy = (y2-y1)/Length.
3) Начальная точка: x = x1 + 0.5 * sign(dx), y = y1 + 0.5 * sign(dy).
4) На каждом шаге: x += dx, y += dy, закрашивается пиксель (int(x), int(y)).

Недостатки: использование операций с плавающей точкой обуславливает малую скорость и возможное накопление ошибки округления.

2.2. Целочисленный алгоритм Брезенхема

Алгоритм Брезенхема (1965 г.) использует только целочисленные операции сложения и вычитания, что делает его значительно быстрее ЦДА. Формирование растрового представления осуществляется движением вдоль основной оси на один пиксель, а изменение другой координаты зависит от значения ошибки.

Принцип работы (для первого октанта):

1) Начальная ошибка: e = 2*Dy - Dx.
2) Если e >= 0: сдвиг по неведущей оси (y += 1), e -= 2*Dx.
3) Всегда: сдвиг по ведущей оси (x += 1), e += 2*Dy.

Обобщение на все октанты выполняется через определение ведущей оси и знаков приращений.

2.3. Алгоритм Ву

Алгоритм Ву обеспечивает сглаживание (anti-aliasing) линий. В отличие от алгоритма Брезенхема, на каждом шаге закрашиваются два пикселя, а интенсивность каждого из них устанавливается пропорционально расстоянию от центра точки до идеальной линии. Сумма интенсивностей двух пикселей всегда равна единице.

Горизонтальные, вертикальные и диагональные линии не требуют сглаживания и рисуются отдельной функцией без распределения интенсивности.

3. Структура программы

Файловая структура проекта:

Файл	Назначение
main.py	Точка входа, запуск приложения
editor.py	Главное окно редактора (меню, тулбар, панели)
algorithms.py	Реализация 3 алгоритмов (ЦДА, Брезенхем, Ву)
canvas_widget.py	Виджет рисования (пользовательский режим)
debug_widget.py	Виджет отладочного режима (сетка + таблица)
requirements.txt	Зависимости (PyQt5)

algorithms.py — каждая функция принимает координаты (x1, y1, x2, y2) и возвращает словарь с ключами: points (список точек для отрисовки с интенсивностью) и steps (данные для отладочной таблицы).

editor.py — главное окно содержит: меню «Отрезки» для выбора алгоритма, панель инструментов с кнопками алгоритмов, переключатель режимов (Пользовательский / Отладочный), панель ввода координат и статусбар.

canvas_widget.py — виджет пользовательского режима: рисование отрезков мышью (два клика), предпросмотр линии, попиксельная отрисовка результата с учётом интенсивности (для Ву).

debug_widget.py — виджет отладочного режима: дискретная сетка с пошаговой визуализацией (закрашенные пиксели с номерами шагов, идеальная линия), таблица параметров, кнопки управления (Сброс / Следующий шаг / Все шаги), масштабирование сетки.

4. Проверка корректности

Для верификации реализации использован тестовый пример из практикума: отрезок из точки (0, 0) в точку (9, 4).

Ожидаемый результат (из таблицы практикума):

Шаг i	0	1	2	3	4	5	6	7	8	9
x	0	1	2	3	4	5	6	7	8	9
y	0	0	1	1	2	2	3	3	4	4

Результат: все три алгоритма (ЦДА, Брезенхем, Ву) дают идентичный набор пикселей для данного отрезка, полностью совпадающий с ожидаемым результатом из практикума.

Дополнительно проверена работа во всех четырёх квадрантах: (0,0)-(8,5), (0,0)-(-8,5), (0,0)-(-8,-5), (0,0)-(8,-5). Результаты корректны.

Пошаговое выполнение алгоритма Брезенхема (0,0) - (9,4):

i	e	x	y	e' (скорр.)	Plot (x, y)

0		0	0	-1	(0, 0)
1	-1	1	0	7	(1, 0)
2	7	2	1	-3	(2, 1)
3	-3	3	1	5	(3, 1)
4	5	4	2	-5	(4, 2)
5	-5	5	2	3	(5, 2)
6	3	6	3	-7	(6, 3)
7	-7	7	3	1	(7, 3)
8	1	8	4	-9	(8, 4)
9	-9	9	4	-1	(9, 4)

5. Демонстрация работы

На рисунке 1 показан пользовательский режим работы редактора: построение отрезка алгоритмом ЦДА.

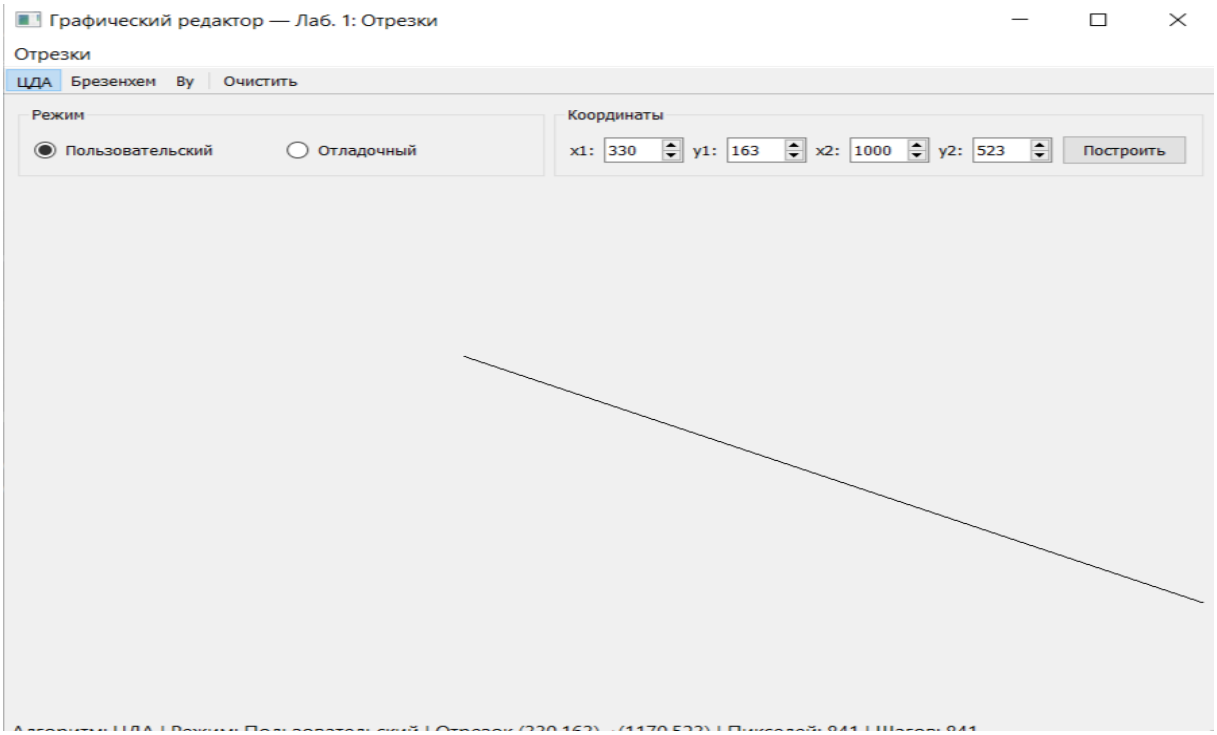

Рис. 1. Пользовательский режим — построение отрезка (ЦДА)

На рисунке 2 показан отладочный режим: пошаговая визуализация алгоритма на дискретной сетке с таблицей параметров.

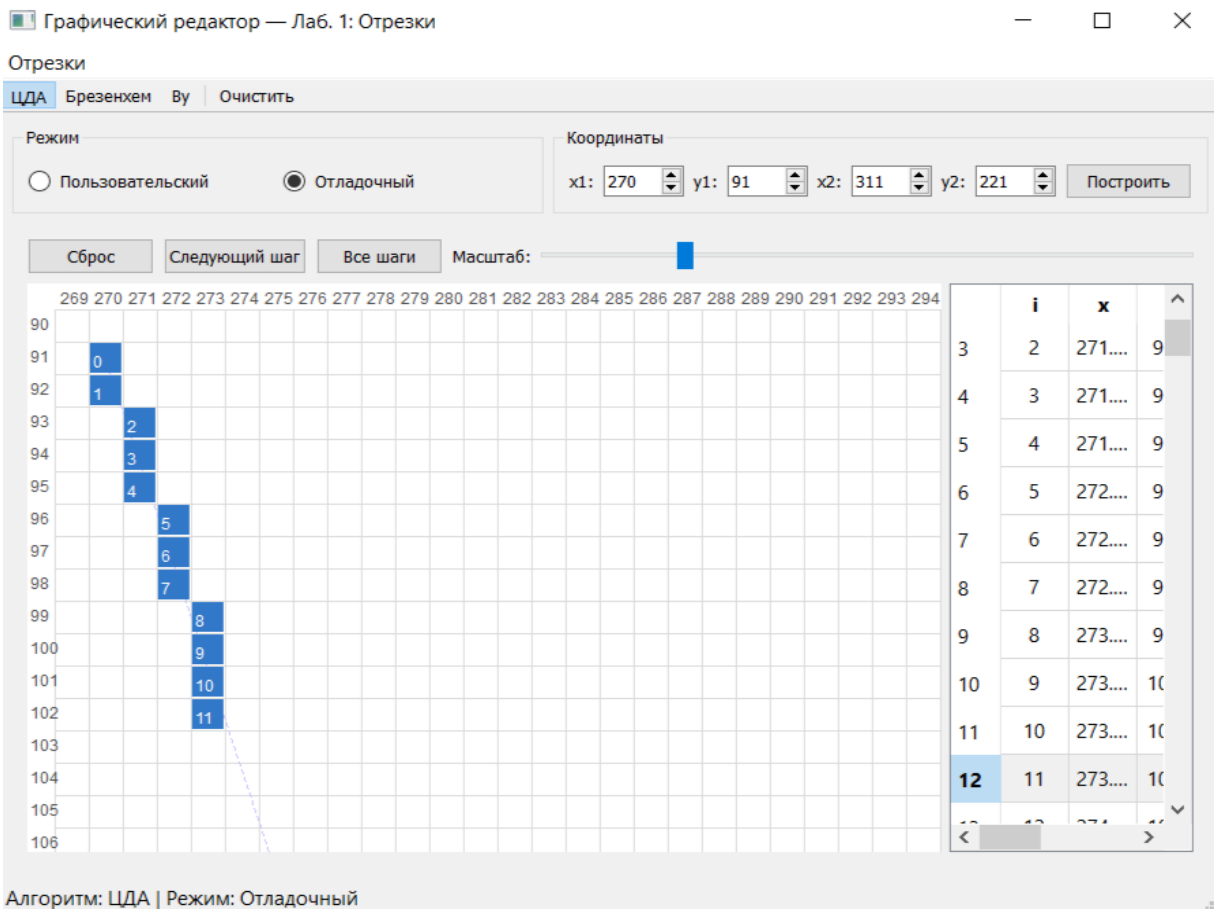

Рис. 2. Отладочный режим — пошаговая визуализация на сетке

6. Инструкция по использованию

Запуск:

```
cd GIIS/lab1 && pip install -r requirements.txt && py -3.11 main.py
```

Выбор алгоритма: через меню «Отрезки» или панель инструментов (кнопки ЦДА / Брезенхем / Ву). Текущий алгоритм отображается в статусбаре.

Пользовательский режим: рисование мышью (два клика: начало и конец отрезка) или ввод координат в поля x1, y1, x2, y2 с последующим нажатием кнопки «Построить».

Отладочный режим: переключение радиокнопкой «Отладочный». Доступно пошаговое выполнение (кнопки Сброс / Следующий шаг / Все шаги), таблица параметров каждого шага, масштабируемая дискретная сетка с пунктирной идеальной линией и закрашенными пикселями.

7. Вывод

В ходе лабораторной работы разработан графический редактор на Python 3.11 с использованием PyQt5, реализующий три алгоритма построения отрезков: ЦДА, целочисленный Брезенхем и Ву (со сглаживанием). Редактор поддерживает два режима работы: пользовательский (рисование мышью и по координатам) и отладочный (пошаговая визуализация на дискретной сетке с таблицей параметров). Корректность реализации

подтверждена сравнением с примерами из практикума.